RICE UNIVERSITY

The GPU Execution Model

Who owns which element, how the GPU hides slow memory, and what launches and copies cost

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 2

Where we are

Last time: the hardware

132 SMs: CUDA and tensor cores

grid $\rightarrow$ block $\rightarrow$ warp $\rightarrow$ thread

SIMT and divergence

blocks run in waves



Today: the execution model

who owns which element

hiding memory latency: occupancy

launches, copies, and streams

timing GPU work honestly

In our pipeline

T3 is due Friday. **T1 and Lab 3** are released after today: you will time kernels and copies with exactly today's rules.

Why do we need an execution model?

The job: $z = x + y$, $n = 10^8$ floats
on a GPU that runs **hundreds of thousands** of threads at once

How do we hand 10^8 little jobs to all those threads?

A. Python loop: one launch per element
10^8 launches $\approx$ **400 s**

B. One launch for all elements
moving the bytes $\approx$ **0.36 ms**

Key idea

B is a **million** times faster: one launch, many threads, each knowing **its own element**.
Every launch and every copy still **costs time**.

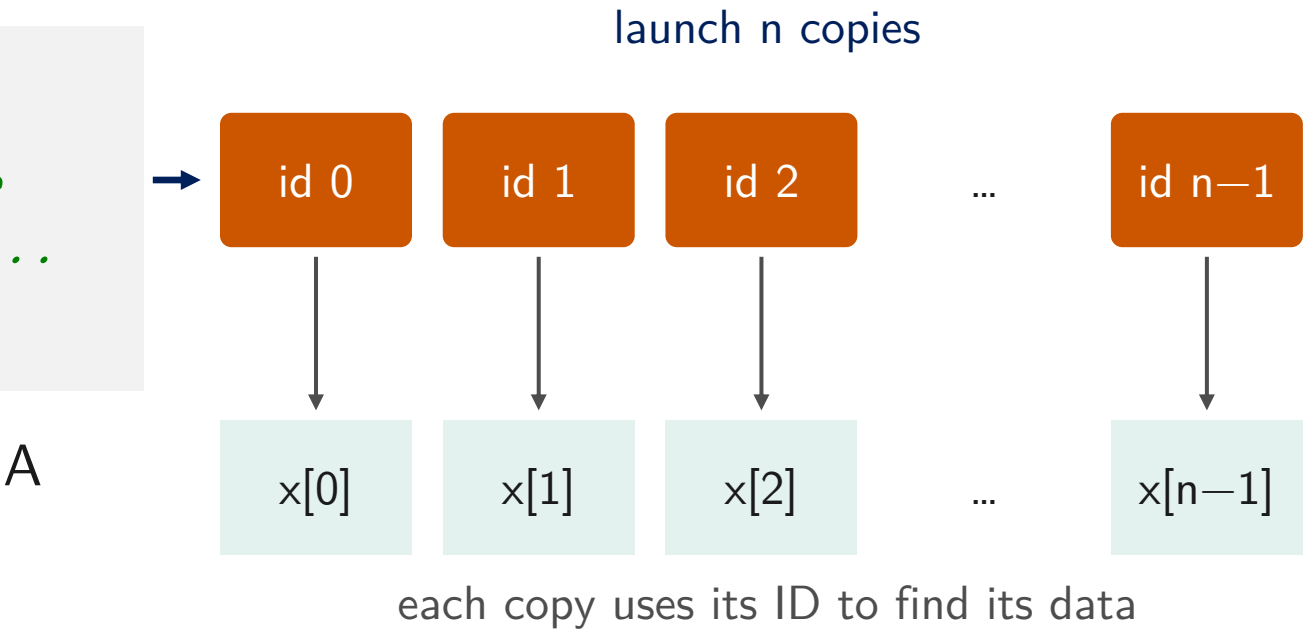
Source: $\approx$ 4 µs per launch: a round assumption, measured in today's activity; H100 datasheet (SXM5).

Kernels and launches, in plain words

ONE KERNEL, WRITTEN ONCE

```
def add(x, y, z):       # pseudocode
    i = my_id()         # 0, 1, 2, ...
    z[i] = x[i] + y[i]
```

A **thread** is one worker with its own ID. A **kernel** is the function every thread runs.



Key idea

You write **one** function; one **launch** runs **many** copies; the **ID** decides the data. CUDA: code for one thread (today). Triton: one block (Week 14).

Where kernels show up

- Every **PyTorch op** is a kernel someone wrote (CUDA or Triton).
- **torch.compile** writes Triton kernels for you (Week 14).
- **FlashAttention**: one hand-written kernel that speeds up large language models.
- **NCCL** collectives that add gradients across GPUs are kernels (Week 8).

The trade-off

You gain: control over bytes; several ops fused into one

You own: indices, races, a busy GPU, honest timing

In our pipeline

Why an execution model? To write kernels that are correct, keep the GPU busy, and are timed honestly. T1 asks for exactly that.

Source: Dao et al., "FlashAttention," NeurIPS 2022; PyTorch documentation, torch.compile (TorchInductor generates Triton).

Learning goals for today

- Compute a thread's **global index** and size a grid with **ceiling division**.
- Explain why threads synchronize only **within a block**, and what **atomics** cost.
- Use **Little's law** and **occupancy** to reason about hiding memory latency.
- Predict the cost of **kernel launches** and **host-to-device copies** with $\alpha + \beta n$.
- **Time** GPU work correctly: synchronize, warm up, report the median.

A puzzle to start

```
t0 = time.perf_counter()
z = x + y                  # x, y: 10**8 floats on GPU
t1 = time.perf_counter()
print((t1 - t0) * 1e3, "ms")    # prints 0.02
```

Bytes ÷ bandwidth

$Q = 1.2$ GB, $\beta = 3.35$ TB/s

$Q / \beta \approx$ **0.36 ms**

The timer says 18× faster than memory allows. What happened?

Watch out

1.2 GB in 0.02 ms would be **60 TB/s** — impossible. The launch returned **before the GPU did the work**: the timer measured the launch, not the kernel.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Part 1

Threads, blocks, and grids

Which element does each thread own — and when must threads wait?

A kernel is written for one thread

```
__global__ void add(const float* x,
                    const float* y,
                    float* z, int n) {
  int i = blockIdx.x * blockDim.x
        + threadIdx.x;
  if (i < n) z[i] = x[i] + y[i];
}

int B = 256;  // threads per block
add<<<(n + B - 1) / B, B>>>(x, y, z, n);
```

- `blockIdx.x`: which block
- `blockDim.x`: threads per block
- `threadIdx.x`: which thread in the block
- `if (i < n)`: guard the tail

Key idea

Write the body for **one** element; the launch runs it for **all** of them.

Source: NVIDIA CUDA C++ Programming Guide, "Programming Model".

The global index

block 0							
0	1	2	3	4	5	6	7

threadIdx.x (inside the block)

block 1							
0	1	2	3	4	5	6	7

block 2							
0	1	2	3	4	5	6	7

0	1	2	3	4	5	6	7

8	9	10	11	12	13	14	15

16	17	18	19	20	21	22	23

global index i (in the whole grid)

$$i = b \cdot B + t, \quad 0 \le t < B$$

$$B = 256, \quad b = 2, \quad t = 5 \quad \Rightarrow \quad i = 517$$

Key idea

Mixed radix: b = i // B and t = i mod B. Every element has **exactly one** owner.

Sizing the grid: ceiling division

$$\text{blocks} = \lceil n \,/\, B \rceil = (n + B - 1) \,//\, B$$

$$n = 1000, \quad B = 256: \quad 4 \text{ blocks} = 1024 \text{ threads}, \quad 24 \text{ idle}$$

$$n = 10^8, \quad B = 1024: \quad 97{,}657 \text{ blocks}, \quad \text{last one } 256 \text{ valid}$$

Watch out

Without the bounds check, the idle threads read and write **past the end** of the array: a crash at best, silent corruption at worst.

Choosing the block size

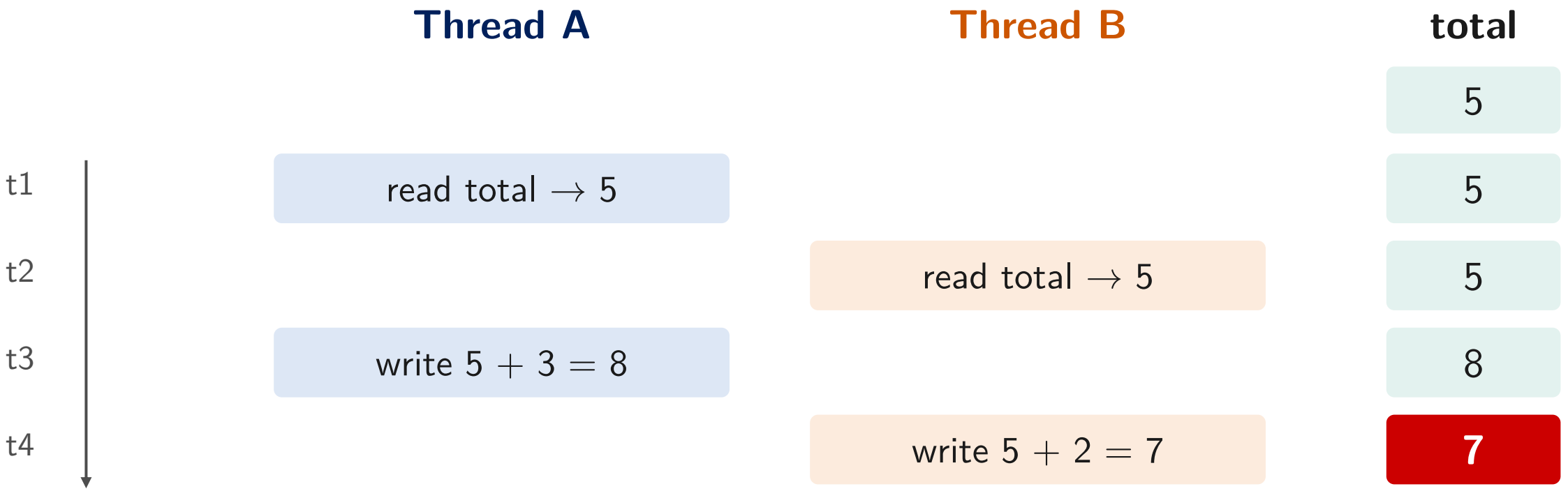
- A multiple of **32**: a block of 100 threads still occupies 4 warps.
- At most **1024** threads per block (CUDA limit).
- Registers and shared memory limit blocks per SM (occupancy, Part 2): start at 128–512.
- Enough blocks: $n = 10^4$, $B = 1024 \rightarrow$ only 10 blocks for 132 SMs.

Key idea

Many blocks, each a multiple of 32 threads — then **measure**; the best size depends on the kernel.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100). H100 whitepaper (132 SMs).

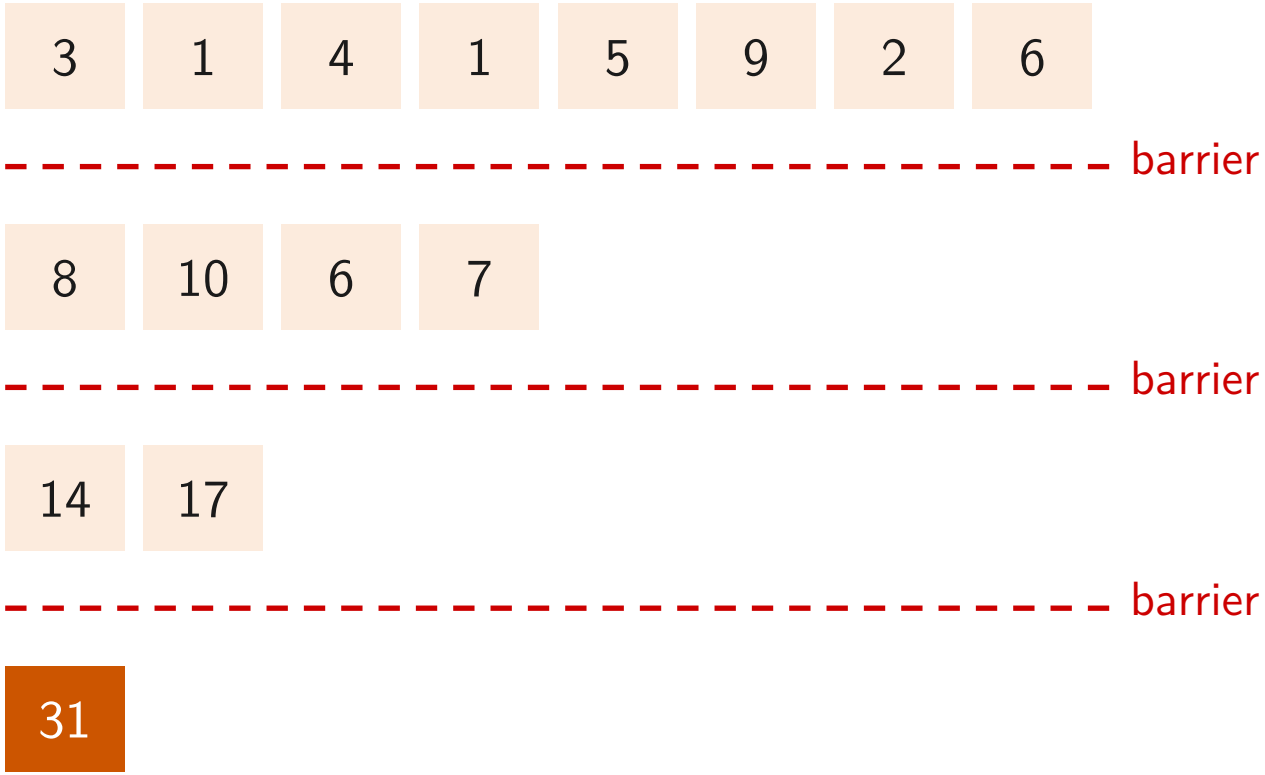
A data race: two threads, one counter



Watch out

Expected 10, got **7**: a lost update. `total += v` is read, add, write — three steps another thread can interleave.

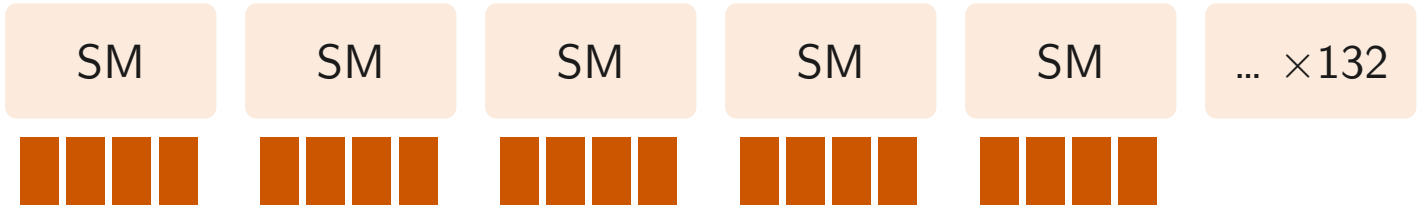
Barriers: waiting within a block



- Step k reads what **other threads** wrote in step $k - 1$.
- 256 values: $\log_2 256 = \mathbf{8}$ steps, 8 barriers.
- Every thread of the block must reach the barrier.
- `torch.sum` does all of this for you.

Source: NVIDIA CUDA C++ Programming Guide, "Synchronization Functions" (__syncthreads).

Why blocks cannot wait for each other



running: $\leq$ 8 blocks of 256 per SM $\rightarrow \leq$ 1,056 blocks



waiting: 390,625 blocks for n $= 10^8 \rightarrow \approx$ 370 waves

Grid barrier = deadlock

Running blocks would wait for blocks that cannot start until they finish.

Key idea

The only grid-wide barrier is the **end of a kernel**: "global max, then normalize" is two launches.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100).

Atomics: indivisible read–modify–write

- `atomicAdd(&total, v)`: read, add, write as **one** step.
- Same-address atomics are **serialized**: one per block, not per element.
- Arrival order is **unspecified**, and float addition is not associative:

$$\text{FP32:} \quad (10^8 + 1) - 10^8 = 0, \qquad (10^8 - 10^8) + 1 = 1$$

Watch out

Atomic sums are correct but **not bitwise reproducible**. Test with tolerances; for exact repeats, sum partials in a fixed order.

Source: IEEE 754 binary32 (spacing 8 near 10^8); CUDA C++ Programming Guide, "Atomic Functions".

Check yourself

Two minutes with a neighbour.

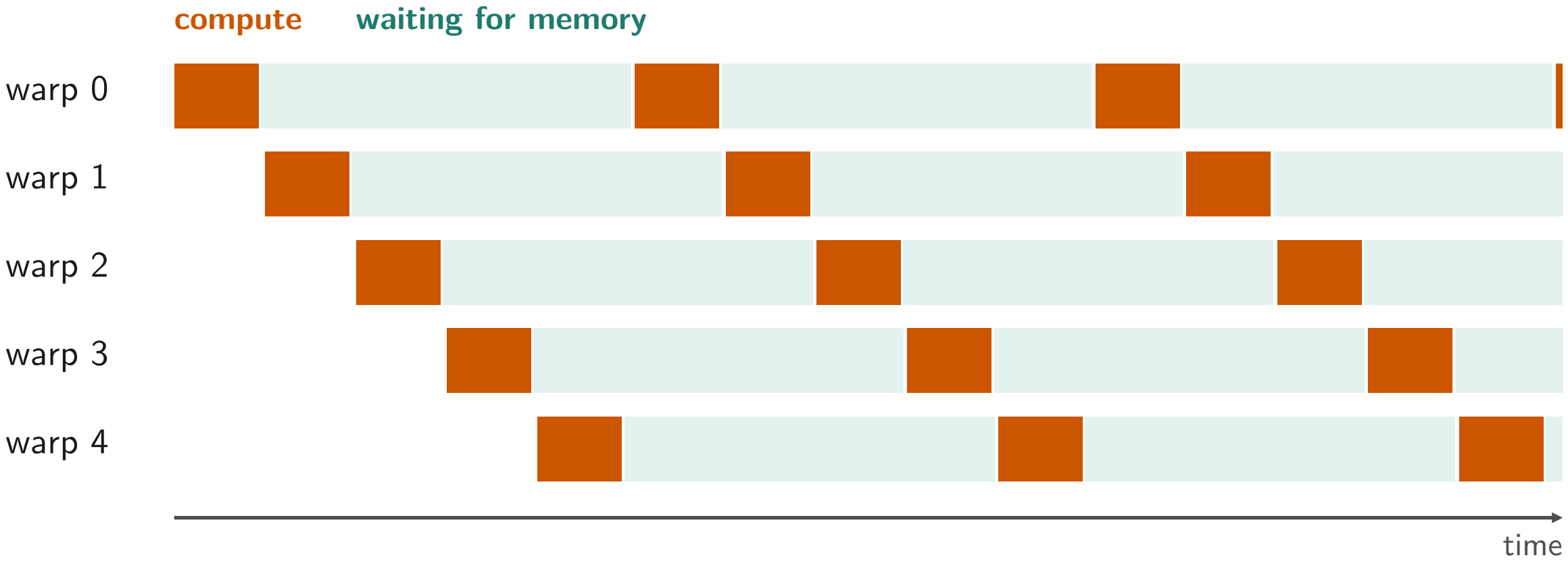
- n $= 5000$, B $= 128$: how many blocks, and how many threads are idle?
- With B $= 128$, which element does thread 37 of block 12 own?
- A kernel needs the max of the whole vector before normalizing it. Why not one kernel with a grid barrier? What instead?

Part 2

Hiding memory latency

What does a warp do while it waits for memory?

Hiding latency by switching warps



Key idea

While warp 0 waits, warps 1–4 compute; switching is free.

Source: latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

Little's law: how much must be in flight?

$$L = \lambda \cdot W$$

$$\text{inside} = \text{rate} \times \text{time each stays}$$

A café

2 customers per minute

each stays 15 minutes

$\rightarrow$ **30** inside on average

$$\text{memory: bytes in flight} = \text{bandwidth} \times \text{latency}$$

Key idea

To keep memory busy, the GPU must have **bandwidth × latency** bytes requested at all times.

Source: J. D. C. Little, "A Proof for the Queuing Formula L = λW," Operations Research 9(3), 1961.

How many threads are enough?

$$\text{in flight} = 3.35\ \text{TB/s} \times 600\ \text{ns} \approx 2\ \text{MB}$$

$$\text{per SM: } 2\ \text{MB} \,/\, 132 \approx 15\ \text{KB}$$

- 4-byte load per thread $\rightarrow \approx$ **3,800** threads per SM needed.
- An SM holds at most **2,048** threads $\rightarrow$ not enough.
- Fix: **16-byte** loads, several in flight per thread.

Key idea

Hiding latency needs **concurrency**: enough bytes in flight to cover the wait.

Source: Little (1961); H100 datasheet (bandwidth); latencies are rounded, from microbenchmarks (e.g. Luo et al., IPDPS 2024).

Occupancy: how many warps fit

Occupancy = resident warps ÷ 64 (the most an H100 SM holds)

Kernel: 256 threads per block · 128 registers per thread · 48 KB shared memory per block

- Registers: 65,536 ÷ 128 = 512 threads = **16 warps**
- Shared memory: $\lfloor 228 \text{ KB} \div 48 \text{ KB} \rfloor = 4$ blocks $\times$ 8 warps = **32 warps**
- Thread limit: 2,048 ÷ 32 = **64 warps**
- Binding constraint: min(16, 32, 64) = 16 warps $\rightarrow$ **25% occupancy**

Key idea

The **tightest** resource sets occupancy. Fewer registers or less shared memory per thread $\rightarrow$ more warps.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100).

Occupancy is a means, not a goal

$$\text{bytes in flight per SM} = \text{threads} \times \text{bytes per thread}$$

$$15\ \text{KB} \approx 2{,}048\ \text{threads} \times 7.4\ \text{bytes} \quad (100\%)$$

$$15\ \text{KB} \approx 512\ \text{threads} \times 30\ \text{bytes} \quad (25\%)$$

Watch out

Low occupancy is fine if each thread keeps **more loads in flight**. Raise occupancy only when the kernel is **waiting on latency**.

Source: Little (1961); H100 datasheet; NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100).

Check yourself

Two minutes with a neighbour.

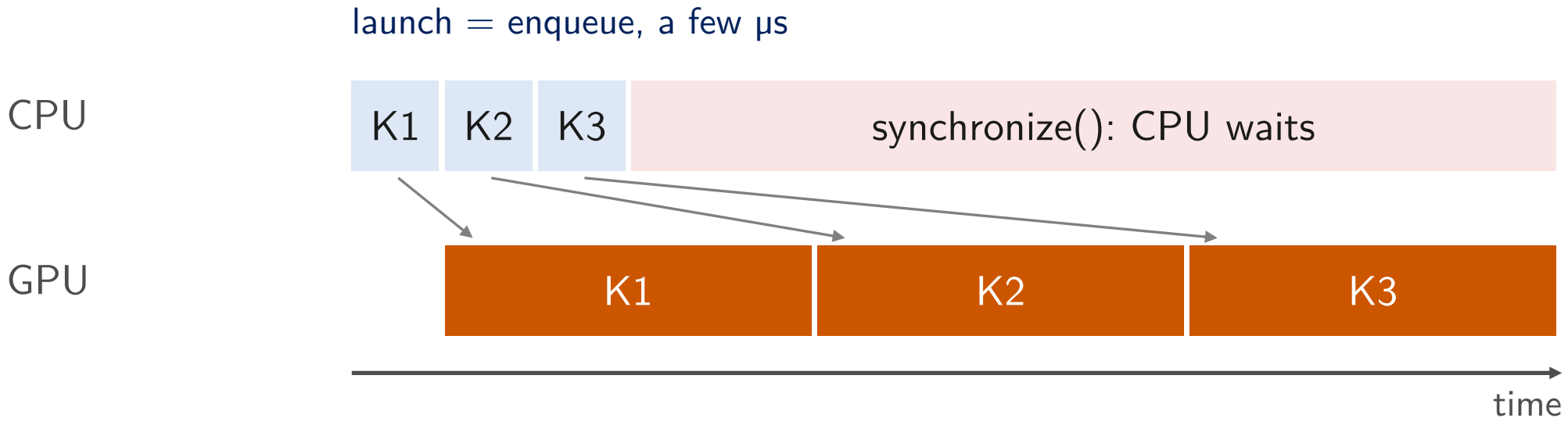
- Bandwidth 2 TB/s, latency 500 ns: how many bytes must be in flight?
- 256 threads per block, 64 registers per thread, no shared memory: what is the occupancy?
- How can a kernel at 25% occupancy still reach full memory bandwidth?

Part 3

Launches, copies, timing

When does GPU work actually run, and what does it cost?

A launch returns immediately



Watch out

Python runs **ahead** of the GPU. A timer without synchronization measures the **launch**, not the kernel — the opening puzzle.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

What a launch costs

$$\text{tiny add, 1,000 floats: } 12 \text{ KB}$$

$$12 \text{ KB} \,/\, 3.35 \text{ TB/s} \approx 3.6 \text{ ns}$$

$$\text{one launch} \approx 4 \,\mu\text{s} \approx 1{,}100\times \text{ longer}$$

When it matters

Hidden while each kernel runs **longer** than a launch: the CPU queues ahead.

Exposed when kernels are **tiny**: the GPU waits for the CPU.

Watch out

Many tiny kernels are **launch-bound**: give each launch more work, or fuse kernels (Week 14).

Source: ≈ 4 µs per launch: assumed round number, measured in today's activity; H100 datasheet (SXM5).

Host-to-device copies: $\alpha + \beta n$ (Week 6)

$$T(n) = \alpha + \beta\ n, \qquad \beta = 1\ /\ \text{bandwidth}$$

$$\text{PCIe 5.0 } \times 16\text{:}\ \ \text{bandwidth} \approx 64\ \text{GB/s per direction}$$

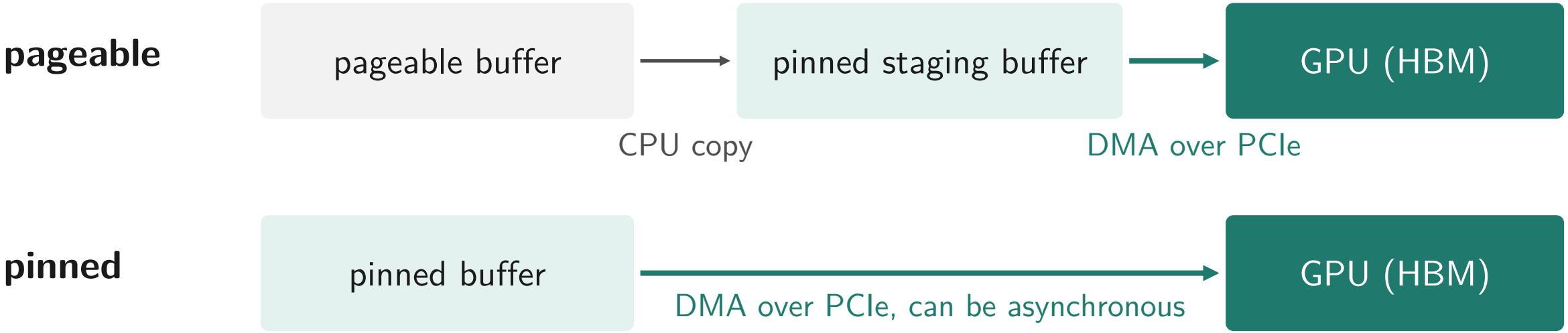
$$n_{1/2} = \alpha\ /\ \beta \approx 10\ \mu\text{s} \times 64\ \text{GB/s} = 640\ \text{KB}$$

In practice

Below $n_{1/2}$ a copy is **latency-bound**: batch small copies. HBM is $\approx$ **52×** faster than PCIe: copy once, keep data on the GPU.

Source: PCI-SIG, PCIe 5.0 $\times 16 \approx 64$ GB/s per direction; H100 datasheet (SXM5); $\alpha \approx 10$ µs assumed.

Pageable versus pinned host memory



`x.pin_memory()` · `DataLoader(pin_memory=True)` · `x.to("cuda", non_blocking=True)`

Key idea

Pageable: **two** copies and the CPU waits. Pinned: **one** copy that can overlap with compute.

Source: NVIDIA CUDA C++ Programming Guide, "Page-Locked Host Memory"; PyTorch "CUDA semantics" (pinned memory).

Hidden synchronization points

Operation	Why the CPU must wait
`torch.cuda.synchronize()`	explicit: wait for all queued work
`x.item()`, `print(x)`, `x.cpu()`	needs the value on the CPU
`if loss > 0:`	Python branch on a GPU value
copy from pageable host memory	the copy must be staged first

Watch out

A stray `.item()` per step drains the queue and leaves the GPU **idle** between kernels.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

Streams: ordered queues of GPU work

```
s = torch.cuda.Stream()                  # 2nd queue
x_cpu = torch.randn(2**24).pin_memory()
with torch.cuda.stream(s):
    x = x_cpu.to("cuda", non_blocking=True)
y = w @ w          # default stream: overlaps
torch.cuda.current_stream().wait_stream(s)
z = 2 * x          # safe: runs after the copy
```

- **Within** a stream: in order
- **Across** streams: may overlap
- Async copy needs **pinned** memory
- `wait_stream`: order across streams

Watch out

Without `wait_stream`, the kernel could read `x` before the copy lands: a **race between streams**.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

Overlapping copy and compute

serial: one stream



overlapped: two streams



$$\text{serial:} \quad n\,(c + k) \;=\; 8 \text{ ms}$$

$$\text{overlapped:} \quad c + (n-1)\cdot \max(c,\, k) + k \;=\; 5 \text{ ms}$$

c: 64 MB over PCIe 5 at $\approx$ 64 GB/s $\approx$ 1 ms; k = 1 ms; n = 4 chunks

Source: PCI-SIG, PCIe 5.0 x16 $\approx$ 64 GB/s per direction; chunk sizes and kernel time are illustrative.

Timing GPU work: synchronize, warm up, median

```
def bench(fn, reps=20):
    fn(); torch.cuda.synchronize()   # warm up
    ts = []
    for _ in range(reps):
        t0 = time.perf_counter()
        fn()
        torch.cuda.synchronize()     # wait for GPU
        ts.append(time.perf_counter() - t0)
    return statistics.median(ts)     # seconds
```

- **Synchronize**: launches are asynchronous.
- **Warm up**: the first call allocates, loads, compiles.
- **Median** of repeats, plus the spread.
- **Convert** to GB/s and compare.

In our pipeline

This is the fix for the opening puzzle: **synchronize before you read the clock**.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

CUDA events: timing on the GPU's clock

```
s = torch.cuda.Event(enable_timing=True)
e = torch.cuda.Event(enable_timing=True)
ts = []
for _ in range(reps):
    s.record(); fn(); e.record()
    e.synchronize()                # wait for e
    ts.append(s.elapsed_time(e))   # ms
ms = statistics.median(ts)
```

- Events are timestamps **in the stream**.
- They measure **GPU time** only.
- Still: warm up, repeat, median.

Key idea

perf_counter + synchronize: **wall time** of the call. Events: **GPU time** between two points. Say which one you report.

Source: PyTorch documentation, torch.cuda.Event and torch.cuda.synchronize.

Check yourself

Two minutes with a neighbour.

- `t0 = perf_counter(); y = f(x); t = perf_counter() - t0` — what does t measure?
- Name two operations in a training loop that silently synchronize.
- $\alpha = 10$ µs, 64 GB/s: predict the copy time of 64 KB and of 64 MB.

IN-CLASS BUILD

Measure a launch and a copy

Predict first, then measure with PyTorch. Work in pairs.

Exercise: predict, then measure

- **Predict** the copy times for 4 KB, 4 MB, 400 MB, and $n_{1/2}$.
- **Measure** one launch: many tiny kernels, one synchronize.
- **Measure** the copies (median of repeats).
- **Fit** α and the bandwidth; compare. Optional: pinned vs pageable.

Copy size	Predicted	Measured
4 KB		
4 MB		
400 MB		
one launch		

USE (round assumptions)

$\alpha \approx 10$ µs · bandwidth ≈ 64 GB/s (PCIe 5.0 $\times 16$) · launch ≈ 4 µs

Source: PCI-SIG, PCIe 5.0 $\times 16$; α and the launch cost are assumed round numbers.

How to measure it

```
x = torch.zeros(1, device="cuda")
torch.cuda.synchronize()
t0 = time.perf_counter()
for _ in range(10_000):
    x.add_(1)                       # tiny kernel
torch.cuda.synchronize()
launch = (time.perf_counter() - t0) / 10_000

for n in (10**3, 10**6, 10**8):   # floats
    h = torch.randn(n)              # pageable
    # h = h.pin_memory()            # optional
    t = bench(lambda: h.to("cuda"))
    print(4 * n, "bytes:", t * 1e6, "us")
```

- 10,000 launches, **one** synchronize
- bench: warm-up, sync, median
- bandwidth = bytes / time
- pinned: uncomment one line

Source: PyTorch documentation: torch.cuda.synchronize, Tensor.pin_memory, Tensor.to.

Exercise: solutions

Copy size	α + n / bandwidth	Regime
4 KB	$10 + 0.06 \approx 10$ µs	latency-bound
4 MB	$10 + 62.5 \approx 72$ µs	bandwidth-bound
400 MB	≈ 6.3 ms	bandwidth-bound

$$n_{1/2} = \alpha \times \text{bandwidth} \approx 640 \text{ KB}$$

$$\text{fit: } \text{bandwidth} \approx (n_2 - n_1) \,/\, (t_2 - t_1), \quad \alpha \approx t_1 - n_1 \,/\, \text{bandwidth}$$

What you should see

Expect less than 64 GB/s, less again from pageable memory. **Explain the gap** between prediction and measurement.

Source: PCI-SIG, PCIe 5.0 ×16; $\alpha = 10$ µs assumed.

Key takeaways

- Global index $\mathbf{i = b \cdot B + t}$; grid $= \lceil \mathbf{n} \,/\, \mathbf{B} \rceil$ blocks; guard the tail with $\mathbf{i < n}$.
- Barriers work **within a block**; across blocks, use **atomics** or a **second kernel**.
- Warps hide memory latency: **bytes in flight = bandwidth $\times$ latency** (Little).
- **Occupancy** is set by the tightest resource; it is a means, not a goal.
- Launches are **asynchronous** and cost µs; copies cost $\alpha + \mathbf{n} \,/\, \mathbf{bandwidth}$; pin to overlap.
- Time GPU work honestly: **synchronize**, **warm up**, report the **median**.

Next time, and readings

When	What
After this lecture	T1 and Lab 3 released; T3 due Friday
Week 13, Lecture 1	The GPU memory hierarchy: registers, shared memory, L2, HBM
Week 13, Lecture 2	Arithmetic intensity and the roofline; T1 checkpoint

READINGS

CUDA C++ Programming Guide: *Hardware Multithreading*; *Asynchronous Concurrent Execution*.

PyTorch documentation: *CUDA semantics* (streams, pinned memory).